

Proyecto #2 — Dulces & Dados

1. Introducción:

A partir del sistema desarrollado en el Proyecto #1 para el Board Game Café "Dulces & Dados", se incorporan nuevos requerimientos funcionales centrados en la gestión de torneos de juegos de mesa. El documento abarca cuatro actividades: la definición de historias de usuario para los tres actores del sistema (administrador, empleado y cliente), la implementación de pruebas automatizadas con JUnit siguiendo TDD, el desarrollo de interfaces de consola independientes para cada tipo de usuario, y la actualización del diseño orientado a objetos para soportar los nuevos requerimientos.

2. Historias de Usuario (HU)

Se presentan las historias de usuario para los tres actores del sistema: Administrador, Empleado y Cliente. Cada historia incluye actor, descripción, precondition, flujo principal, resultado esperado, entradas y salidas.

2.1 Administrador

El administrador es registrado en el sistema de fábrica (login: admin, password: admin123 por defecto al crear nuevo café). Es el único que puede crear torneos y registrar empleados.

HU-A01 — Crear torneo amistoso	
Actor:	Administrador
Descripción:	Como administrador quiero crear un torneo amistoso asociado a un juego del inventario, en un día de la semana específico, con un porcentaje de bono de descuento como premio para el ganador.
Precondición:	El administrador ha iniciado sesión. Hay al menos un juego en el inventario de préstamos con copias disponibles.
Flujo principal:	1. El administrador selecciona 'Gestionar torneos' → 'Crear torneo amistoso'. 2. Ingresar el nombre del torneo. 3. Selecciona el día de la semana (LUNES a DOMINGO). 4. Selecciona el juego de la lista (se muestran juegos con copias disponibles). 5. Ingresar el número máximo de participantes. 6. Ingresar el porcentaje del bono de descuento (1-100). 7. El sistema valida el nombre único, que el inventario tenga copias suficientes y confirma la creación.
Resultado esperado:	El torneo queda registrado con cupos totales, cupos para fanáticos ($\text{ceil}(\text{max} * 0.20)$) y cupos regulares calculados. Aparece en la lista de torneos.
Entradas:	Nombre del torneo (String único), día de semana (opción 1-7), juego (seleccionado de lista), número máximo de participantes (entero > 0), porcentaje de bono (entero 1-100).

Salidas / Resultado:	Confirmación del torneo creado con nombre, día, juego, cupos totales y descripción del premio.
-----------------------------	--

HU-A02 — Crear torneo competitivo

Actor:	Administrador
Descripción:	Como administrador quiero crear un torneo competitivo con tarifa de entrada, de modo que el premio en metálico se financie con las inscripciones pagadas por los clientes.
Precondición:	El administrador ha iniciado sesión. Hay al menos un juego disponible en el inventario de préstamos.
Flujo principal:	1. El administrador selecciona 'Gestionar torneos' → 'Crear torneo competitivo'. 2. Ingresa nombre, selecciona día, elige juego y número máximo de participantes. 3. Ingresa la tarifa de entrada en pesos colombianos (valor > 0). 4. El sistema valida los datos y registra el torneo.
Resultado esperado:	El torneo competitivo queda registrado con tarifa de entrada. El premio en metálico se calcula dinámicamente según los cupos de clientes inscritos (empleados no cuentan).
Entradas:	Nombre del torneo (String único), día de semana, juego, número máximo de participantes (entero > 0), tarifa de entrada (double > 0).
Salidas / Resultado:	Confirmación del torneo creado con tarifa registrada y cupos calculados.

HU-A03 — Ver listado de torneos

Actor:	Administrador
Descripción:	Como administrador quiero ver todos los torneos registrados con su estado de cupos para hacer seguimiento.
Precondición:	El administrador ha iniciado sesión. Existe al menos un torneo registrado.
Flujo principal:	1. El administrador selecciona 'Gestionar torneos' → 'Ver torneos'. 2. El sistema muestra la lista completa con: nombre, tipo, juego, día, cupos fanáticos, cupos regulares disponibles, cupos totales disponibles y descripción del premio.
Resultado esperado:	Lista completa de torneos con estado de cupos actualizado.
Entradas:	Ninguna (solo navegación de menú).
Salidas / Resultado:	Lista de todos los torneos con información completa.

HU-A04 — Registrar empleado

Actor:	Administrador
Descripción:	Como administrador quiero registrar nuevos empleados (meseros o cocineros) con su turno asignado para que puedan acceder al sistema.

Precondición:	El administrador ha iniciado sesión. El login del nuevo empleado no existe en el sistema.
Flujo principal:	1. El administrador selecciona 'Gestionar empleados' → 'Registrar empleado'. 2. Ingresa login y contraseña. 3. Selecciona tipo: Cocinero (1) o Mesero (2). 4. Selecciona el día de turno (LUNES a DOMINGO). 5. El sistema valida que el login no esté en uso y registra al empleado.
Resultado esperado:	El nuevo empleado queda registrado y puede autenticarse desde el portal de empleado.
Entradas:	Login (String único), contraseña (String no vacía), tipo (Cocinero/Mesero), día de turno.
Salidas / Resultado:	Confirmación de registro exitoso con los datos del empleado creado.

HU-A05 — Entregar premio y calcular premio competitivo	
Actor:	Administrador
Descripción:	Como administrador quiero declarar el ganador de un torneo para asignar el bono (amistoso) o calcular el premio en metálico (competitivo).
Precondición:	El administrador ha iniciado sesión. Existe al menos un torneo con inscripciones.
Flujo principal:	1. El administrador selecciona 'Gestionar torneos' → 'Entregar premio amistoso' o 'Calcular premio competitivo'. 2. Selecciona el torneo. 3. Ingresa el login del ganador. 4. El sistema verifica que esté inscrito. 5. Asigna el bono al cliente (amistoso) o calcula y muestra el premio acumulado (competitivo). Los empleados no reciben premio en metálico.
Resultado esperado:	Bono de descuento asignado al cliente ganador (amistoso) o premio en metálico calculado y mostrado (competitivo).
Entradas:	Selección del torneo, login del ganador.
Salidas / Resultado:	Confirmación de bono asignado o monto del premio en metálico.

2.2 Empleado

Los empleados son registrados por el administrador. Cada empleado tiene un día de turno asignado (turnoSemanal). Pueden inscribirse a torneos solo en días distintos a su turno. En torneos competitivos participan gratis pero sin premio en metálico.

HU-E01 — Autenticarse en el sistema	
Actor:	Empleado
Descripción:	Como empleado quiero autenticarme con mi login y contraseña para acceder a las funcionalidades del sistema.

Precondición:	El empleado ha sido registrado previamente por el administrador.
Flujo principal:	1. El empleado ingresa su login y contraseña en el portal de empleado. 2. El sistema valida las credenciales. 3. Si son correctas, muestra el menú principal con el turno asignado visible.
Resultado esperado:	Acceso al menú de empleado. Si las credenciales son incorrectas, se permite hasta 3 intentos.
Entradas:	Login (String), contraseña (String).
Salidas / Resultado:	Acceso al menú o mensaje de error. Cierre tras 3 intentos fallidos.

HU-E02 — Inscribirse a un torneo

Actor:	Empleado
Descripción:	Como empleado quiero inscribirme a un torneo en un día en que no tenga turno asignado, participando gratuitamente.
Precondición:	El empleado ha iniciado sesión. Existen torneos con cupos disponibles. El empleado no tiene turno el día del torneo.
Flujo principal:	1. El empleado selecciona 'Ver torneos disponibles'. 2. Selecciona 'Inscribirme en un torneo'. 3. El sistema muestra los torneos disponibles con su tipo y cupos. 4. El empleado selecciona el torneo e ingresa el número de participantes (1-3). 5. El sistema valida que no tenga turno ese día (comparación directa del día del turno vs. día del torneo). 6. Si pasa la validación, lo inscribe sin cobrar tarifa. Si es competitivo, informa que no recibirá premio en metálico.
Resultado esperado:	El empleado queda inscrito. En torneos competitivos no paga tarifa y no suma al fondo del premio.
Entradas:	Selección del torneo, número de participantes (1-3).
Salidas / Resultado:	Confirmación de inscripción o error si tiene turno ese día.

HU-E03 — Desinscribirse de un torneo

Actor:	Empleado
Descripción:	Como empleado quiero retirar mi inscripción de un torneo antes de que se realice.
Precondición:	El empleado ha iniciado sesión y está inscrito en al menos un torneo.
Flujo principal:	1. El empleado selecciona 'Desinscribirse de un torneo'. 2. El sistema muestra los torneos en los que está inscrito con la cantidad de cupos. 3. El empleado selecciona el torneo del que desea retirarse. 4. El sistema elimina la inscripción y libera el cupo.
Resultado esperado:	El empleado ya no aparece inscrito en ese torneo y el cupo queda disponible.
Entradas:	Selección del torneo (de lista de inscritos).

Salidas / Resultado:	Confirmación de desinscripción exitosa.
-----------------------------	---

2.3 Cliente

Los clientes pueden registrarse por su propia cuenta desde el portal de cliente. Pueden inscribir hasta 3 participantes por torneo, marcar juegos como favoritos y acumular bonos de descuento de distintos torneos amistosos ganados.

HU-C01 — Registrarse en el sistema	
Actor:	Cliente (sin cuenta)
Descripción:	Como usuario nuevo quiero crear mi propia cuenta para poder acceder a los servicios del Board Game Café.
Precondición:	No existe una cuenta con el mismo login en el sistema.
Flujo principal:	1. En la pantalla de inicio, el usuario selecciona 'Registrarme'. 2. Ingresa un login. 3. El sistema verifica que el login no esté en uso. 4. Si está libre, ingresa su contraseña. 5. El sistema crea la cuenta de cliente y guarda el estado.
Resultado esperado:	La cuenta queda creada. El cliente puede iniciar sesión de inmediato con sus credenciales.
Entradas:	Login (String, único en el sistema), contraseña (String no vacía).
Salidas / Resultado:	Confirmación de registro exitoso. Error si el login ya existe o si la contraseña está vacía.

HU-C02 — Inscribirse a un torneo	
Actor:	Cliente
Descripción:	Como cliente quiero inscribirme a un torneo del catálogo, pudiendo registrar hasta 3 participantes, accediendo primero a los cupos de fanáticos si el juego está en mis favoritos.
Precondición:	El cliente ha iniciado sesión. Existen torneos con cupos disponibles.
Flujo principal:	1. El cliente selecciona 'Inscribirme en un torneo'. 2. El sistema muestra los torneos disponibles con cupos, tipo y premio. 3. El cliente selecciona un torneo e ingresa el número de participantes (1-3). 4. El sistema verifica si el juego está en los favoritos del cliente (esFanaticoDeJuego). 5. Si es fanático y hay cupos de fanáticos disponibles, los usa primero; si se agotan, toma cupos regulares. 6. El sistema registra la inscripción y confirma el tipo de cupo asignado.
Resultado esperado:	El cliente queda inscrito. Los cupos correspondientes (fanáticos y/o regulares) quedan descontados.
Entradas:	Selección del torneo, número de participantes (entero entre 1 y 3).

Salidas / Resultado:	Confirmación con tipo de cupo asignado. Error si ya está inscrito, no hay cupos o el número es inválido.
-----------------------------	--

HU-C03 — Desinscribirse de un torneo

Actor:	Cliente
Descripción:	Como cliente quiero retirar mi inscripción de un torneo, liberando todos los cupos que tomé.
Precondición:	El cliente ha iniciado sesión y está inscrito en al menos un torneo.
Flujo principal:	1. El cliente selecciona 'Desinscribirse de un torneo'. 2. El sistema muestra los torneos en que está inscrito con el número de cupos totales. 3. El cliente selecciona el torneo del que se quiere retirar. 4. El sistema elimina TODOS sus cupos (fanáticos + regulares) de ese torneo.
Resultado esperado:	Todos los cupos del cliente en ese torneo son liberados. El cliente ya no aparece inscrito.
Entradas:	Selección del torneo (de lista de inscritos).
Salidas / Resultado:	Confirmación de desinscripción exitosa con cupos liberados.

HU-C04 — Agregar juego a favoritos

Actor:	Cliente
Descripción:	Como cliente quiero marcar juegos como favoritos para acceder a los cupos reservados para fanáticos en los torneos de esos juegos.
Precondición:	El cliente ha iniciado sesión. Existen juegos en el inventario de préstamos. El juego no está ya en los favoritos del cliente.
Flujo principal:	1. El cliente selecciona 'Mis juegos favoritos'. 2. Selecciona 'Agregar juego favorito'. 3. El sistema muestra la lista de juegos del inventario de préstamos. 4. El cliente selecciona un juego. 5. El sistema lo agrega a su lista de favoritos (sin duplicados).
Resultado esperado:	El juego queda registrado en la lista de favoritos del cliente.
Entradas:	Selección del juego (de la lista del inventario).
Salidas / Resultado:	Confirmación de que el juego fue agregado a favoritos. Lista actualizada de favoritos.

HU-C05 — Ver y usar bono de descuento

Actor:	Cliente
Descripción:	Como cliente ganador de un torneo amistoso quiero consultar mis bonos activos acumulados de distintos torneos.

Precondición:	El cliente ha iniciado sesión y tiene al menos un bono de descuento activo.
Flujo principal:	1. El cliente selecciona 'Mis bonos de descuento'. 2. El sistema muestra la lista numerada de bonos con su porcentaje. 3. El cliente ve el porcentaje de descuento de cada bono disponible. 4. Los bonos se consumen al momento de realizar una compra (el sistema los aplica por índice).
Resultado esperado:	Lista de bonos activos con su porcentaje. Cada bono puede usarse en una compra diferente.
Entradas:	Ninguna (solo navegación de menú).
Salidas / Resultado:	Lista de bonos de descuento activos del cliente.

3: Pruebas Automatizadas con JUnit

Las pruebas se implementaron siguiendo TDD (Test-Driven Development): primero se escribieron las pruebas y luego la implementación. Se utilizó JUnit 4. Se construyeron pruebas unitarias para la lógica de torneos y pruebas de integración basadas en las historias de usuario.

3.1 Pruebas Unitarias — GestorTorneosTest.java

Estas pruebas verifican la lógica interna de GestorTorneos, Torneo, TorneoAmistoso y TorneoCompetitivo de forma unitaria. El setUp() inicializa un Cafe con dos juegos (Catan y Uno) en inventario de préstamos, tres clientes (cliente1, cliente2, clienteFanatico con Catan como favorito) y dos empleados (empleadoLibre con turno SABADO y empleadoEnTurno con turno LUNES).

Nombre del test	Qué valida	Clase bajo prueba
testCrearTorneoAmistoso_exitoso	Crea torneo amistoso con nombre, día, juego, cupos y porcentaje de descuento correctos.	GestorTorneos
testCrearTorneoCompetitivo_exitoso	Crea torneo competitivo con tarifa correcta y esAmistoso()=false.	GestorTorneos
testCrearTorneo_nombreDuplicado_falla	Retorna null si ya existe un torneo con el mismo nombre.	GestorTorneos
testCrearTorneo_sinInventario_falla	Retorna null si el juego no tiene inventario de préstamos.	GestorTorneos
testCrearTorneo_participantesMayorQueMaxJugadores_conInventarioSuficiente	Permite crear torneo con más participantes que	GestorTorneos

	maxJugadores si hay copias suficientes.	
testCrearTorneo_tarifaEntradaCero_falla	Retorna null si la tarifa de entrada del torneo competitivo es 0.	GestorTorneos
testCuposFanaticos_calculoCorrecto	getCuposFanaticos() retorna $\text{ceil}(10 * 0.20) = 2$.	Torneo
testCuposFanaticos_redondeoHaciaArriba	getCuposFanaticos() retorna $\text{ceil}(8 * 0.20) = 2$.	Torneo
testCuposFanaticos_fanaticoPrimeroUsaCupoReservado	Un fanático al inscribirse con 1 cupo usa el cupo de fanáticos: cuposFanaticosOcupados=1, cuposOcupados=0.	Torneo
testCuposFanaticos_cuandoSeAgotan_fanaticTomaRegular	Fanático con 2 cupos en torneo de 5 (1 cupo fanático): toma 1 fanático + 1 regular.	Torneo
testCuposNoFanatico_soloTomaCuposRegulares	Cliente no fanático inscrito con 2 cupos: cuposFanaticosOcupados=0, cuposOcupados=2.	Torneo
testInscribirCliente_exitoso	Cliente se inscribe exitosamente con 2 cupos. yaInscrito() retorna true.	GestorTorneos
testInscribirCliente_masDe3Cupos_falla	inscribirCliente con cantidad=4 retorna false.	GestorTorneos
testInscribirCliente_yaInscrito_falla	Inscribir al mismo cliente dos veces en el mismo torneo retorna false.	GestorTorneos
testInscribirCliente_sinCuposDisponibles_falla	Inscribir cuando el torneo está lleno retorna false.	GestorTorneos
testDesinscribirCliente_liberaTodosLosCupos	Tras desinscribir, yaInscrito()=false y cuposOcupados=0.	GestorTorneos
testDesinscribirCliente_noInscrito_falla	Desinscribir usuario no inscrito retorna false.	GestorTorneos
testDesinscripcion_fanatico_liberaCuposFanaticos	Tras desinscribir un fanático, cuposFanaticosOcupados vuelve a 0.	GestorTorneos

testInscribirEmpleado_libre_exitoso	Empleado con turno SABADO se inscribe en torneo del LUNES. Éxito.	GestorTorneos
testInscribirEmpleado_conTurno_falla	Empleado con turno LUNES no puede inscribirse en torneo del LUNES. Retorna false.	GestorTorneos
testInscribirEmpleado_diaDiferente_exitoso	Empleado con turno LUNES se inscribe en torneo del MARTES. Éxito.	GestorTorneos
testEntregarBonoAmistoso_asignaCorrectamente	entregarBonoAmistoso() agrega bono del 20% a la lista de bonos del cliente.	GestorTorneos
testEntregarBonoAmistoso_acumulaBonosDistintosTorneos	Ganar dos torneos distintos acumula dos bonos con distintos porcentajes.	GestorTorneos
testUsarBonoDescuento_loConsume	usarPrimerBonoDescuento() retorna 0.10 y deja lista vacía.	Cliente
testEntregarBono_clienteNoInscrito_falla	entregarBonoAmistoso a cliente no inscrito retorna false.	GestorTorneos
testPremioCompetitivo_calculoCorrectoConClientes	2 clientes (2+1 cupos) con tarifa 15000: premio = 45000.	TorneoCompetitivo
testPremioCompetitivo_empleadoNoRecibePremio	calcularPremioCompetitivo con empleado como ganador retorna 0.0.	GestorTorneos
testPremioCompetitivo_empleadoNoContaEnRecaudado	1 cliente (1 cupo) + 1 empleado, tarifa 10000: premio = 10000 (empleado no cuenta).	TorneoCompetitivo
testMultiplesTorneos_simultaneos	Crear 3 torneos distintos: cafe.getTorneos().size() = 3.	GestorTorneos
testCliente_puedeInscribirseEnVariosTorneos	Un cliente puede inscribirse en múltiples torneos distintos simultáneamente.	GestorTorneos
testEmpleadoTieneTurnoEnDia_mismo_dia	empleadoTieneTurnoEnDia(empleado, 'LUNES') = true cuando turno es LUNES.	GestorTorneos
testEmpleadoTieneTurnoEnDia_dia_diferente	empleadoTieneTurnoEnDia(empleado, 'MARTES') = false cuando turno es LUNES.	GestorTorneos

testCuposTotalesDisponibles_seReducenAlInscribir	cuposTotalesDisponibles() se reduce al inscribir participantes.	Torneo
testDesinscripcion_restauraCuposDisponibles	Tras desinscripción, cuposTotalesDisponibles() vuelve al valor original.	Torneo

3.2 Pruebas de Integración — TorneoIntegracionTest.java

Estas pruebas cubren flujos completos del sistema, verificando la interacción entre GestorTorneos, Torneo, Cliente, Empleado e InscripcionTorneo.

Nombre del test de integración	Flujo que valida	Historia de usuario
testHistoriaCompletaTorneoAmistoso	Admin crea torneo amistoso. Fanático se inscribe (cupo fanático). Cliente normal se inscribe (cupo regular). Admin entrega bono al ganador. Bono queda en lista del cliente.	HU-A01, HU-C02, HU-A05
testHistoriaTorneoCompetitivoConEmpleado	Admin crea torneo competitivo. Empleado sin turno se inscribe gratis. Cliente paga. premio = solo cupos de cliente $\times$ tarifa. Empleado no recibe premio.	HU-A02, HU-E02, HU-A05
testInscripcionYDesinscripcionLiberaCupos	Cliente se inscribe con 3 cupos. cuposTotalesDisponibles baja. Tras desinscripción, cupos vuelven al total original y yaInscrito = false.	HU-C02, HU-C03

4: Interfaces de Consola

Se implementaron tres aplicaciones de consola independientes dentro del mismo proyecto Eclipse, cada una con su propio método main. Todos los usuarios deben autenticarse para usar las funcionalidades. Los datos se cargan automáticamente desde data/cafe_inicial.dat al iniciar y se guardan en el mismo archivo al salir (opción Salir del menú).

4.1 Estructura de Clases de Interfaz

Clase	Responsabilidad
-------	-----------------

MainAdministrador.java	Punto de entrada del administrador. Carga datos desde cafe_inicial.dat, autentica (hasta 3 intentos) y delega al menú principal.
MainEmpleado.java	Punto de entrada del empleado. Carga datos, autentica (hasta 3 intentos) y muestra el turno del empleado en el menú.
MainCliente.java	Punto de entrada del cliente. Permite registro propio o login, luego muestra el menú principal del cliente.

4.2 Menú del Administrador — MainAdministrador.java

Opción	Funcionalidad
1 Gestionar empleados	Registrar empleado (login, password, tipo Cocinero/Mesero, día de turno) o ver lista de empleados con tipo y turno.
2 Gestionar inventario	Agregar juego a préstamo (nombre, año, empresa, categoría, min/max jugadores, edad, dificultad, cantidad) o agregar producto de cafetería (bebida/pastelería).
3 Gestionar torneos	Crear torneo amistoso, crear torneo competitivo, ver torneos, ver inscripciones de un torneo, entregar premio amistoso, calcular premio competitivo.
4 Gestionar turnos	Ver turnos, ver solicitudes pendientes, aprobar o rechazar solicitud de cambio de turno.
5 Salir	Guarda todos los datos en data/cafe_inicial.dat y cierra la aplicación.

4.3 Menú del Empleado — MainEmpleado.java

Opción	Funcionalidad
1 Ver torneos disponibles	Muestra todos los torneos con nombre, día, tipo y cupos totales disponibles.
2 Inscribirme en un torneo	Selecciona torneo e ingresa número de participantes (1-3). Valida que no tenga turno ese día.
3 Desinscribirme de un torneo	Muestra torneos en los que está inscrito con cupos tomados. Al seleccionar, libera todos sus cupos.
4 Solicitar cambio de turno	Lista otros empleados disponibles para solicitar intercambio de turno.
5 Préstamo de juego	Inicia préstamo de juego (solo si no está en turno activo).
6 Salir	Guarda todos los datos en data/cafe_inicial.dat y cierra la aplicación.

4.4 Menú del Cliente — MainCliente.java

Opción	Funcionalidad
1 Registrarme / 2 → Iniciar sesión	Desde la pantalla inicial: registro con login y password únicos, o autenticación hasta 3 intentos.
1 Ver torneos disponibles	Muestra torneos con nombre, día, tipo, cupos disponibles y descripción del premio.
2 Inscribirme en un torneo	Ingresa número de participantes (1-3). Sistema aplica cupo fanático si el juego está en favoritos.
3 Desinscribirme de un torneo	Lista torneos donde está inscrito con cupos tomados. Al seleccionar, libera todos sus cupos.
4 Mis bonos de descuento	Lista numerada de bonos activos con su porcentaje de descuento.
5 Mis juegos favoritos	Ver favoritos actuales y agregar nuevos desde el inventario de préstamos.
6 Salir	Guarda todos los datos en data/cafe_inicial.dat y cierra la aplicación.

4.5 Validaciones de Entrada Implementadas

Cada clase Main implementa sus propios métodos leerEntero(min, max) y leerDouble(min, max) con bucle de reintento ante entradas inválidas. Adicionalmente se implementan validaciones lógicas en GestorTorneos y GestorUsuarios:

Método / Validación	Qué valida	Error si...
leerEntero(min, max)	Entero dentro del rango [min, max]	No es número o está fuera del rango; reintenta en bucle
leerDouble(min, max)	Número decimal en [min, max]	No es número o está fuera del rango; reintenta en bucle
Login no vacío	String que no sea vacío ni solo espacios	isEmpty() = true; método retorna antes de procesar
Password no vacía	String que no sea vacío ni solo espacios	isEmpty() = true; método retorna antes de procesar

Nombre torneo único	Login no existente en el sistema	cafe.buscarTorneoPorNombre(nombre) != null → retorna null
Login empleado/cliente único	gestorUsuarios.registrarEmpleado/Cliente retorna null si login ya existe	Login ya registrado
Tarifa de entrada > 0	tarifaEntrada > 0 en torneo competitivo	tarifaEntrada <= 0 → retorna null
Inventario suficiente	ceil(numParticipantes / maxJugadores) ≤ copias disponibles	No hay copias suficientes → retorna null
Cupos ≤ 3 por usuario	cantidad entre 1 y 3 al inscribirse	cantidad < 1 cantidad > 3 → retorna false
Usuario no inscrito dos veces	torneo.yaInscrito(usuario) antes de inscribir	yaInscrito = true → retorna false
Empleado sin turno ese día	empleadoTieneTurnoEnDia(empleado, diaTorneo)	turno.equals(diaTorneo) = true → retorna false

4.6 Persistencia de Datos

Al iniciar cada aplicación se carga automáticamente el archivo data/cafe_inicial.dat usando serialización Java (clase Persistencia). Al seleccionar Salir se sobrescribe el mismo archivo con el estado actualizado. No se le pide al usuario el nombre del archivo en ningún momento. El directorio data/ se crea automáticamente si no existe.

5: Diseño Actualizado del Sistema

Se presentan los cambios introducidos al diseño del Proyecto #1 para soportar los nuevos requerimientos de torneos.

5.1 Nuevas Clases

Clase	Tipo	Descripción
Torneo	Abstracta	Define atributos comunes: nombre, diaSemana, juego, numParticipantes, cuposFanaticos (ceil(max*0.20)), cuposOcupados, cuposFanaticosOcupados, inscripciones (List<InscripcionTorneo>). Implementa inscribir(), desinscribir(), yaInscrito(), buscarInscripcion(), cuposTotalesDisponibles().
TorneoAmistoso	Concreta	Extiende Torneo. Agrega porcentajeDescuento (double, ej. 0.15 = 15%). Descripción del premio: 'Bono de descuento del X% en la siguiente compra'.

TorneoCompetitivo	Concreta	Extiende Torneo. Agrega tarifaEntrada (double) y premioMetalico. Método calcularPremio() que suma tarifa × cupos de inscripciones donde el usuario es instancia de Cliente.
InscripcionTorneo	Concreta	Registra la inscripción de un usuario: usuario (Usuario), cuposFanaticos (int), cuposRegulares (int). totalCupos() = cuposFanaticos + cuposRegulares.
GestorTorneos	Lógica	Encapsula toda la lógica de torneos: crearTorneoAmistoso(), crearTorneoCompetitivo(), inscribirCliente(), inscribirEmpleado(), desinscribirCliente(), desinscribirEmpleado(), entregarBonoAmistoso(), calcularPremioCompetitivo(), empleadoTieneTurnoEnDia(), mostrarTorneos(), mostrarInscripciones().

5.2 Cambios a Clases Existentes del Proyecto #1

Clase	Cambio realizado	Justificación
Cliente	+ juegosFavoritos: List<Juego> + bonosDescuento: List<Double> + agregarJuegoFavorito(Juego) + esFanaticoDeJuego(Juego): boolean + agregarBonoDescuento(double) + usarPrimerBonoDescuento(): double + usarBonoDescuento(int): double + tieneBonosDescuento(): boolean	Necesario para verificar si es fanático del juego del torneo (cupos preferencial) y para acumular bonos de distintos torneos amistosos ganados.
Empleado	+ turnoSemana: String (día exacto: 'LUNES', 'MARTES', etc.)	Necesario para validar si el empleado tiene turno el día en que se realiza el torneo (comparación directa de strings en mayúsculas).
Cafe	+ torneos: List<Torneo> + agregarTorneo(Torneo) + getTorneos(): List<Torneo> + buscarTorneoPorNombre(String): Torneo	El Café es el punto central del sistema y gestiona la colección de torneos. Se mantiene coherencia con el patrón de diseño existente.

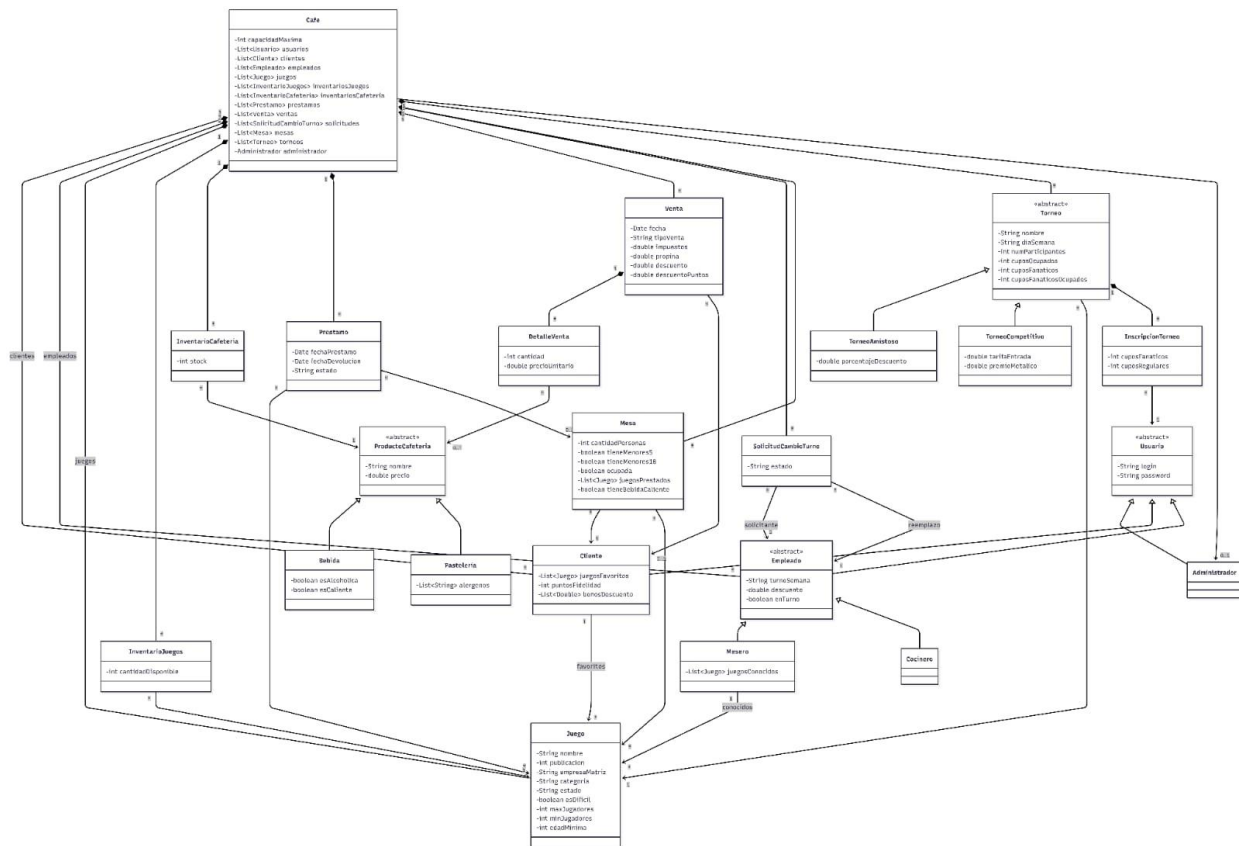
4.3 Atributos Detallados — Clase Torneo (abstracta)

Atributo / Método	Tipo	Descripción
nombre	String	Nombre identificador del torneo (único en el sistema)
diaSemana	String	Día en que se realiza, almacenado en mayúsculas: LUNES, MARTES... DOMINGO
juego	Juego	Juego asociado al torneo (del inventario de préstamos)

numParticipantes	int	Capacidad total de cupos del torneo
cuposFanaticos	int	Math.ceil(numParticipantes * 0.20) — calculado en constructor
cuposOcupados	int	Cupos regulares tomados actualmente
cuposFanaticosOcupados	int	Cupos de la zona fanáticos tomados actualmente
inscripciones	List<InscripcionTorneo>	Lista de todas las inscripciones al torneo
totalCuposOcupados()	int	cuposFanaticosOcupados + cuposOcupados
cuposTotalesDisponibles()	int	numParticipantes – totalCuposOcupados()
cuposRegularesDisponibles()	int	(numParticipantes – cuposFanaticos) – cuposOcupados
cuposFanaticosDisponibles()	int	cuposFanaticos – cuposFanaticosOcupados

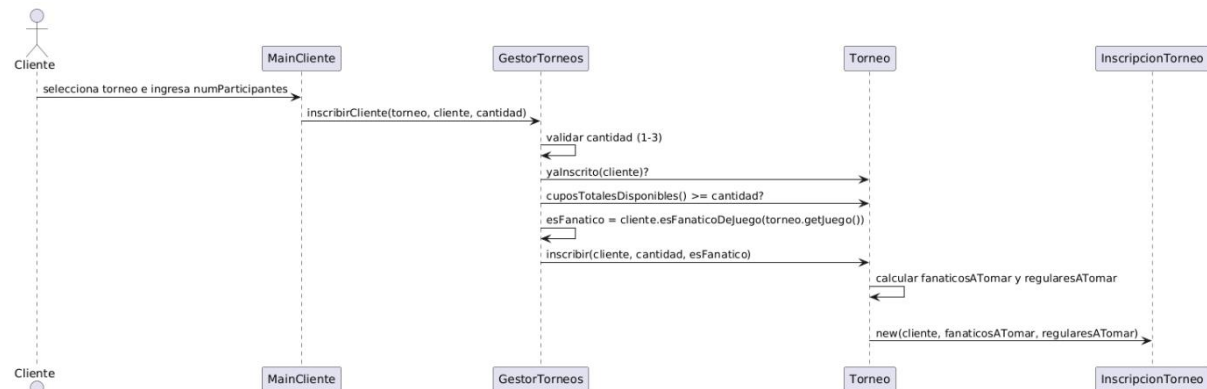
5.4 Diagrama de Clases UML

A continuación se presenta el diagrama de clases actualizado del sistema, incluyendo todas las nuevas clases y relaciones introducidas en el Proyecto #2:



5.5 Diagramas de Secuencia — Nuevas Funcionalidades

5.5.1 Inscripción de cliente a torneo



@startuml

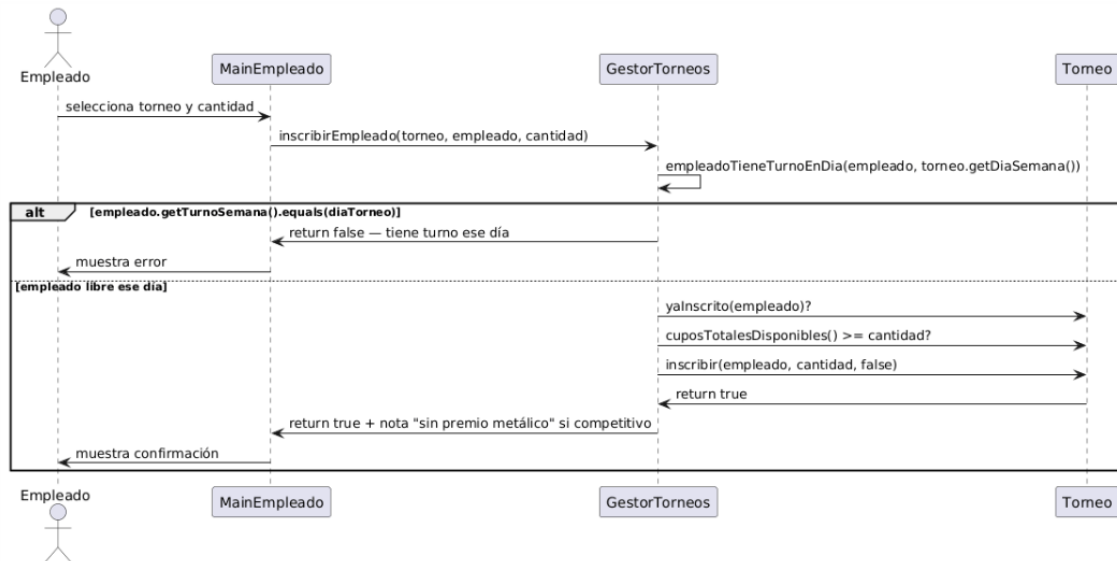
actor Cliente

participant "MainCliente" as MC


```
participant "GestorTorneos" as GT
participant "Torneo" as T
participant "InscripcionTorneo" as IT

Cliente -> MC : selecciona torneo e ingresa numParticipantes
MC -> GT : inscribirCliente(torneo, cliente, cantidad)
GT -> GT : validar cantidad (1-3)
GT -> T : yaInscrito(cliente)?
GT -> T : cuposTotalesDisponibles() >= cantidad?
GT -> GT : esFanatico = cliente.esFanaticoDeJuego(torneo.getJuego())
GT -> T : inscribir(cliente, cantidad, esFanatico)
T -> T : calcular fanaticosATomar y regularesATomar
alt cupos fanáticos disponibles y esFanatico
    T -> IT : new(cliente, fanaticosATomar, regularesATomar)
else solo cupos regulares
    T -> IT : new(cliente, 0, cantidad)
else sin cupos disponibles
    T -> MC : return false
end
T -> GT : return true
GT -> MC : mostrar confirmación con tipo de cupo
MC -> Cliente : muestra resultado
@enduml
```

5.5.2 Inscripción de empleado a torneo



@startuml

actor Empleado

participant "MainEmpleado" as ME

participant "GestorTorneos" as GT

participant "Torneo" as T

Empleado -> ME : selecciona torneo y cantidad

ME -> GT : inscribirEmpleado(torneo, empleado, cantidad)

GT -> GT : empleadoTieneTurnoEnDia(empleado, torneo.getDiaSemana())

alt empleado.getTurnoSemana().equals(diaTorneo)

GT -> ME : return false — tiene turno ese día

ME -> Empleado : muestra error

else empleado libre ese día

GT -> T : yaInscrito(empleado)?

GT -> T : cuposTotalesDisponibles() >= cantidad?

GT -> T : inscribir(empleado, cantidad, false)

T -> GT : return true

GT -> ME : return true + nota "sin premio metálico" si competitivo

ME -> Empleado : muestra confirmación

end

@enduml

5.6 Resumen de Cambios vs Proyecto #1

Elemento	Estado Proyecto #1	Estado Proyecto #2
Clase Torneo	No existía	Nueva clase abstracta con subclases TorneoAmistoso y TorneoCompetitivo
TorneoAmistoso / Competitivo	No existía	Nuevas clases concretas con lógica de premio diferenciada
InscripcionTorneo	No existía	Nueva clase de asociación con cuposFanaticos + cuposRegulares
GestorTorneos	No existía	Nueva clase de lógica con todos los RF del proyecto 2
Cliente.juegosFavoritos	No existía	Nueva List<Juego>, sin duplicados
Cliente.bonosDescuento	No existía	Nueva List<Double>, acumulable entre distintos torneos
Empleado — validación de turno	booleano enTurno simple	String turnoSemana con día específico, comparado contra diaSemana del torneo
Cafe.torneos	No existía	Nueva List<Torneo> con métodos agregarTorneo y buscarTorneoPorNombre
Interfaces de consola	No implementadas	3 main independientes: MainAdministrador, MainEmpleado, MainCliente
Pruebas JUnit	No implementadas	34 pruebas unitarias (GestorTorneosTest) + 3 integración (TorneoIntegracionTest) con TDD

6. Conclusión

El desarrollo del Proyecto #2 permitió aplicar de manera integral las etapas del ciclo de vida del software, desde el análisis de requerimientos hasta la implementación y pruebas. La incorporación del módulo de torneos implicó extender el diseño existente mediante herencia y composición, introduciendo las clases Torneo, TorneoAmistoso, TorneoCompetitivo e InscripcionTorneo. La aplicación de TDD garantizó que cada requerimiento funcional tuviera cobertura antes de su implementación, resultando en un sistema más robusto y verificable. Las interfaces de consola desarrolladas para cada actor del sistema validan correctamente las entradas del usuario y persisten el estado del café entre sesiones, cumpliendo con todos los requerimientos establecidos en el enunciado.

Anexo A: Declaración de Uso de IAG

Conforme a los lineamientos de la Universidad de los Andes (2024) y las instrucciones del curso ISIS-1226, se declara el uso de herramientas de IAG bajo el Nivel 3 autorizado para este proyecto. El Nivel 3 permite usar IAG para editar y refinar ideas, siempre que se documente cómo se usó.

A.1 Usos Aplicados bajo Nivel 3

- Generación de getters y setters para las clases Torneo, TorneoAmistoso, TorneoCompetitivo e InscripcionTorneo (equivalente a lo que hace Eclipse automáticamente con Source → Generate Getters and Setters).
- Generación de constructores básicos de las nuevas clases.
- Creación de la plantilla base del Scanner para lectura de consola con manejo de InputMismatchException y ciclo de reintento.
- Refinamiento de mensajes de error para mejorar claridad al usuario en los métodos de consola.
- Corrección de errores de compilación puntuales (depuración de imports y tipos).

A.2 Lo que NO se hizo con IAG

- La lógica de inscripción/desinscripción, el cálculo de cupos fanáticos (ceil 20%) y la distribución entre cupos fanáticos y regulares fue desarrollada manualmente.
- Las reglas de negocio (turno de empleado, cálculo del premio competitivo, acumulación de bonos) fueron implementadas por el equipo.
- El diseño de clases, atributos, relaciones y el patrón de gestor fue realizado por el grupo sin asistencia directa de IAG.
- Las pruebas JUnit y la estrategia TDD (orden de escritura prueba → implementación) fueron diseñadas por el equipo.

A.3 Colección de Prompts Utilizados

#	Prompt	Propósito
1	Genera getters y setters para esta clase Java con estos atributos: nombre, diaSemana, numParticipantes, cuposFanaticos, cuposOcupados, inscripciones	Getters/setters Torneo
2	Genera getters y setters para InscripcionTorneo con: usuario, cuposFanaticos, cuposRegulares	Getters/setters InscripcionTorneo
3	Crea la plantilla base de un Scanner para leer entrada de consola con manejo de NumberFormatException y ciclo de reintento	Plantilla leerEntero() en Main classes
4	El siguiente mensaje de error es confuso para el usuario, ¿cómo lo redactarías mejor?: [mensaje original]	Refinamiento mensajes de consola

Todos los elementos de lógica de negocio, diseño y pruebas fueron desarrollados directamente por los integrantes del grupo.